Quantum Computing and Quantum Machine Learning

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo[1]

March 5, 2025

Plans for the week of March 3-7

1. Reminder on basics of the VQE method and how to perform measurements for the simpler one- and two-qubit Hamiltonians
2. How to perform measurements
3. Simulating efficiently Hamiltonians on quantum computers with the VQE method and gradient descent to optimize the state function ansatz
4. Introducing the Lipkin model
5. Work on project 1

Meet the VQE

The Variational quantum eigensolver (VQE) is a hybrid quantum-classical algorithm that finds the smallest eigenvalue (and corresponding eigenvector) of a given Hamiltonian. One of the main applications of the algorithm is finding ground state energy of quantum mechanical systems. It has a big advantage over the quantum phase estimation (QPE) algorithms, that also can be used for finding the ground state energy of a molecule.

Main advantage

The main advantage is that VQE uses much smaller circuit depths (or gates) than QPE, what is very important for NISQ (Noisy Intermediate-Scale Quantum) era quantum computation. In the NISQ era (now!) we are working with qubits that are very noisy because they are not isolated from the environment well enough. Thus, there is small and finite time to work with qubits until they will be *spoiled*, because of the environment, imperfect gates and etc. This restriction gives a big advantage to those algorithms (like VQE) that are using small depth circuits.

Basic idea

The idea of the VQE algorithm is as follows. We have a Hamiltonian that can be expressed by the sum of tensor products of Pauli operators (Pauli terms):

$$H = 0.4 \cdot IX + 0.6 \cdot IZ + 0.8 \cdot XY.$$

For a given $|\psi\rangle$ we want to measure the expectation value of the Hamiltonian:

$$\langle H \rangle = \langle \psi | H | \psi \rangle = 0.4 \cdot \langle \psi | IX | \psi \rangle + 0.6 \cdot \langle \psi | IZ | \psi \rangle + 0.8 \cdot \langle \psi | XY | \psi \rangle .$$

VQE and quantum circuits

How one can see the $\langle H \rangle$ expectation value could be computed by adding the expectation values of its parts (Pauli terms). The algorithm does exactly that. It constructs a quantum circuit for each Pauli term and computes the expectation value of the corresponding Pauli term. Then, the algorithm sums all calculated expectation values of Pauli terms and obtains the expectation value of H. In this algorithm, we will do this routine of estimating the expectation value of H over and over again for different trial wavefunctions (ansatz states) $|\psi\rangle$.

Varying paramters

It is known that the eigenvector $|\psi_g\rangle$ that minimizes the expectation value $\langle H \rangle$ corresponds to the eigenvector of H that has the smallest eigenvalue. So, basically we can try all possible trial wavefunctions $|\psi\rangle$s to find the $|\psi_g\rangle$ that has the smallest expectation value. Here the question is how we create those trial states?

Constructing trial states

In the algorithm, the trial states are created from a parametrized circuit. By changing the parameters one obtains different wavefunctions (ansatz states). If your circuit with its parameters is good enough you will have access to the subspace of the states that includes the $|\psi_g\rangle$. Otherwise, if the circuit will not have a possibility to generate our desired $|\psi_g\rangle$ it will be impossible to find the right solution.

Hybrid opeations

The parameters of the state preparation circuit are controlled by a classical computer. At each step, the classical computer will change the parameters by using some optimization method in order to create an ansatz state that will have a smaller expectation value then previous ansatz states had. This way the classical computer and the quantum computer are working together to archive the goal of the algorithm (to find the ground state energy). That's way, VQE is a quantum-classical hybrid algorithm.

Reminder on technicalities

From our earlier discussions we know that the Pauli Z matrix has the above basis states as eigen states through

$$\boldsymbol{Z}|0\rangle = +1|0\rangle,$$

and

$$\boldsymbol{Z}|1\rangle = -1|1\rangle,$$

with eigenvalue -1.

Pauli X reminder

For the Pauli X matrix on the other hand we have

$$\boldsymbol{X}|0\rangle = +1|1\rangle,$$

and

$$\boldsymbol{X}|1\rangle = -1|0\rangle,$$

with eigenvalues 1 in both cases. The latter two equations tell us that the computational basis we have chosen, and in which we will prepare our states, is not an eigenbasis of the $\boldsymbol{X}$ matrix.

Rewriting the Pauli X matrix

We rewrite the Pauli X matrix in terms of a Pauli Z matrix using the Hadamard matrix twice, that is

$$\boldsymbol{X} = \boldsymbol{HZH}.$$

The Pauli Y matrix

The Pauli Y matrix can be written as

$$\boldsymbol{Y} = \boldsymbol{H}\boldsymbol{S}^{\dagger}\boldsymbol{Z}\boldsymbol{H}\boldsymbol{S},$$

where S is the phase matrix

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}.$$

Rotations

Another important set of gates are the **rotation operators** R_x, R_y and R_z. By application to a qubit, we can reach any point on the Bloch sphere by usage of all three once. They are expressed as

$$R_x(\theta) = \exp -iX\theta/2 = \begin{bmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{bmatrix},$$

$$R_y(\theta) = \exp -iY\theta/2 = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ -\sin(\theta/2) & \cos(\theta/2) \end{bmatrix},$$

$$R_z(\theta) = \exp -iZ\theta/2 = \begin{bmatrix} \exp -i\theta/2 & 0 \\ 0 & \exp i\theta/2 \end{bmatrix}$$

with all having a period of 4π.

Rayleigh-Ritz variational principle

The Rayleigh-Ritz variational principle states that for a given Hamiltonian H, the expectation value of a trial state or just ansatz $|\psi\rangle$ puts a lower bound on the ground state energy E_0.

$$\frac{\langle\psi|H|\psi\rangle}{\langle\psi|\psi\rangle} \geq E_0.$$

The ansatz

The ansatz is typically chosen to be a parameterized superposition of basis states that can be varied to improve the energy estimate, $|\psi\rangle \equiv |psi(\boldsymbol{\theta})\rangle$ where $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_M)$ are the M optimization parameters.

Rotations again

To have any flexibility in the ansatz $|\psi\rangle$, we need to allow for parametrization. The most common approach is the so-called R_y ansatz, where we apply chained operations of rotating around the y-axis by $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_Q)$ of the Bloch sphere and CNOT operations.
Applications of y rotations specifically ensures that our coefficients always remain real, which often is satisfactory when dealing with many-body systems.

Measurements and more

After the ansatz has been constructed, the Hamiltonian must be applied. As discussed, the Hamiltonian must be written in terms of Pauli strings.
To obtain the expectation value of the ground state energy, one can measure the expectation value of each Pauli string,

$$E(\boldsymbol{\theta}) = \sum_i w_i \langle \psi(\boldsymbol{\theta}) | P_i | \psi(\boldsymbol{\theta}) \rangle \equiv \sum_i w_i f_i,$$

where f_i is the expectation value of the Pauli string i.

Collecting data

This is estimated statistically by considering measurements in the appropriate basis of the operator in the Pauli string.
With N_0 and N_1 as the number of 0 and 1 measurements respectively, we can estimate f_i since

$$f_i = \lim_{N \to \infty} \frac{N_0 - N_1}{N},$$

where N as the number of shots (measurements).
Each Pauli string requires it own circuit, where multiple measurements of each string is required. Adding the results together with the corresponding weights, the ground state energy can be estimated. To optimize with respect to $\boldsymbol{\theta}$, a classical optimizer is often applied.

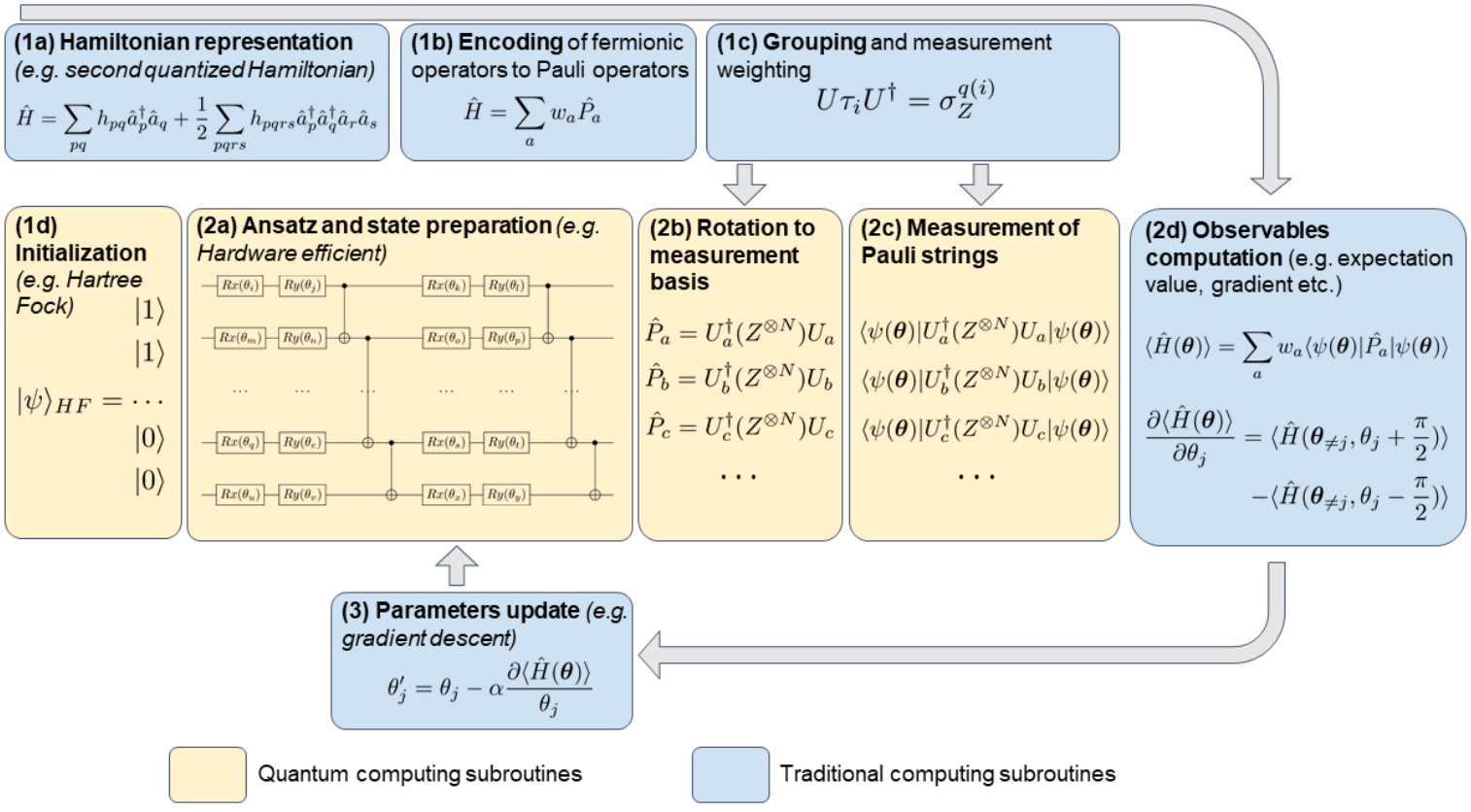
The VQE algoritm

The VQE algorithm consists of several steps, partially done on a classical computer:

1. A parameterized ansatz for the quantum state is implemented on a quantum computer.
2. The ansatz is measured in a given measurement basis.
3. Postprocessing on a classical computer converts the measurement outcomes to an expectation value.
4. Classical minimization algorithms are used to up- date the variational parameters.

The updated variational parameters are then sent back to the quantum computer, and the process is repeated until the optimal variational parameters are found.

VQE overview



Ansatzes

Every possible qubit wavefunction $|\psi\rangle$ can be presented as a vector:

$$|\psi\rangle = \begin{pmatrix} \cos(\theta/2) \\ e^{i\varphi} \cdot \sin(\theta/2) \end{pmatrix},$$

where the numbers θ and φ define a point on the unit three-dimensional sphere, the so-called Bloch sphere.
For a random one qubit Hamiltonian, a *good* quantum state preparation circuit should be able to generate all possible states in the Bloch sphere.

Preparing the states

Before quantum state preparation, our qubit is in the $|0\rangle$ state. This corresponds to the vertical position of the vector in the Bloch sphere. In order to generate any possible $|\psi\rangle$ we will apply $R_x(t_1)$ and $R_y(t_2)$ gates on the $|0\rangle$ initial state

$$R_y(\phi)R_x(\theta)\,|0\rangle = |\psi\rangle\,.$$

The rotation $R_x(\theta)$ corresponds to the rotation in the Bloch sphere around the x-axis and $R_y(\phi)$ the rotation around the y-axis.

Rotations used

These two gates with there parameters (θ and ϕ) will generate for us the trial (ansatz) wavefunctions. The two parameters will be in control of the Classical Computer and its optimization model.

Implementing using qiskit

```
import numpy as np
from random import random
from qiskit import *
def quantum_state_preparation(circuit, parameters):
    q = circuit.qregs[0] # q is the quantum register where the info ab
    circuit.rx(parameters[0], q[0]) # q[0] is our one and only qubit X
    circuit.ry(parameters[1], q[0])
    return circuit
```

VQE and efficient computations of gradients

We start with a reminder on the VQE method with applications to the one-qubit system discussed last week.
Here we revisit the one-qubit system and develop a VQE code for studying this system using gradient descent as a method to optimize the variational ansatz.
We start with a simple 2×2 Hamiltonian matrix expressed in terms of Pauli X and Z matrices, as discussed in the project text.

Symmetric matrix

We define a symmetric matrix $H \in \mathbb{R}^{2\times 2}$

$$H = \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix},$$

We let $H = H_0 + H_I$, where

$$H_0 = \begin{bmatrix} E_1 & 0 \\ 0 & E_2 \end{bmatrix},$$

is a diagonal matrix. Similarly,

$$H_I = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix},$$

where V_{ij} represent various interaction matrix elements.

Non-interacting solution

We can view H_0 as the non-interacting solution

$$H_0|0\rangle = E_1|0\rangle, \tag{1}$$

and

$$H_0|1\rangle = E_2|1\rangle, \tag{2}$$

where we have defined the orthogonal computational one-qubit basis states $|0\rangle$ and $|1\rangle$.

Rewriting with Pauli matrices

We rewrite H (and H_0 and H_I) via Pauli matrices

$$H_0 = \mathcal{E} I + \Omega \sigma_z, \quad \mathcal{E} = \frac{E_1 + E_2}{2}, \; \Omega = \frac{E_1 - E_2}{2},$$

and

$$H_I = c\boldsymbol{I} + \omega_z \sigma_z + \omega_x \sigma_x,$$

with $c = (V_{11} + V_{22})/2$, $\omega_z = (V_{11} - V_{22})/2$ and $\omega_x = V_{12} = V_{21}$. We let our Hamiltonian depend linearly on a strength parameter λ

$$H = H_0 + \lambda H_{\mathrm{I}},$$

with $\lambda \in [0, 1]$, where the limits $\lambda = 0$ and $\lambda = 1$ represent the non-interacting (or unperturbed) and fully interacting system, respectively.

Selecting parameters

The model is an eigenvalue problem with only two available states. Here we set the parameters $E_1 = 0$, $E_2 = 4$, $V_{11} = -V_{22} = 3$ and $V_{12} = V_{21} = 0.2$.
The non-interacting solutions represent our computational basis. Pertinent to our choice of parameters, is that at $\lambda \geq 2/3$, the lowest eigenstate is dominated by $|1\rangle$ while the upper is $|0\rangle$. At $\lambda = 1$ the $|0\rangle$ mixing of the lowest eigenvalue is 1% while for $\lambda \leq 2/3$ we have a $|0\rangle$ component of more than 90%. The character of the eigenvectors has therefore been interchanged when passing $z = 2/3$. The value of the parameter V_{12} represents the strength of the coupling between the two states.

Setting up the matrix

This part is best seen using the jupyter-notebook

```
from  matplotlib import pyplot as plt
import numpy as np
dim = 2
Hamiltonian = np.zeros((dim,dim))
e0 = 0.0
e1 = 4.0
Xnondiag = 0.20
Xdiag = 3.0
Eigenvalue = np.zeros(dim)
# setting up the Hamiltonian
Hamiltonian[0,0] = Xdiag+e0
Hamiltonian[0,1] = Xnondiag
Hamiltonian[1,0] = Hamiltonian[0,1]
Hamiltonian[1,1] = e1-Xdiag
# diagonalize and obtain eigenvalues, not necessarily sorted
EigValues, EigVectors = np.linalg.eig(Hamiltonian)
permute = EigValues.argsort()
EigValues = EigValues[permute]
# print only the lowest eigenvalue
print(EigValues[0])
```

Now rewrite it in terms of the identity matrix and the Pauli matrix X and Z

```
# Now rewrite it in terms of the identity matrix and the Pauli matrix
X = np.array([[0,1],[1,0]])
```

Measurements and computational basis

We have seen how to rewrite the above 2×2 eiegenvalue problem in terms of a Hamiltonian defined by Pauli $\boldsymbol{X}$ and $\boldsymbol{Z}$ matrices, and the identity matrix $\boldsymbol{I}$. Let us make this Hamiltonian that involves only one qubit somewhat more general

$$\langle H \rangle = \langle \psi | H | \psi \rangle = a \cdot \langle \psi | I | \psi \rangle + b \cdot \langle \psi | Z | \psi \rangle + c \cdot \langle \psi | X | \psi \rangle + d \cdot \langle \psi | Y | \psi \rangle .$$

Expectation value of $\boldsymbol{I}$

For the $\boldsymbol{I}$ operator the expectation value is always unity:

$$\langle\psi|\,\boldsymbol{I}\,|\psi\rangle = \langle\psi|\,|\psi\rangle = 1.$$

Its contribution to the overall expectaction value is thus given by the constant a.

The Pauli matrices

For rest of the Pauli operators, we should make the following remark: every one qubit quantum state $|\psi\rangle$ can be represented via different sets of basis vectors:

$$|\psi\rangle = c_1^z \cdot |0\rangle + c_2^z \cdot |1\rangle = c_1^x \cdot |+\rangle + c_2^x \cdot |-\rangle = c_1^y \cdot |+i\rangle + c_2^y \cdot |-i\rangle \, .$$

In more detail

We have

Z eigenvectors $\qquad |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$

For the other two matrices

X eigenvectors $\quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$

Y eigenvectors $\quad |+i\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}, \quad |-i\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix}.$

Analyzing these equations

The first presented eigenvectors for each Pauli has an eigenvalue equal to $+1$: $Z\left|0\right\rangle = +1\left|0\right\rangle$, $X\left|+\right\rangle = +1\left|+\right\rangle$, $Y\left|+i\right\rangle = +1\left|+i\right\rangle$. And the second presented eigenvectors for each Pauli has an eigenvalue equal to -1: $Z\left|1\right\rangle = -1\left|1\right\rangle$, $X\left|-\right\rangle = -1\left|-\right\rangle$, $Y\left|-i\right\rangle = -1\left|-i\right\rangle$. Now, let's calculate the expectation values of these Pauli operators:

$$\left\langle\psi\right| Z \left|\psi\right\rangle = \left(c_1^{z*}\cdot\left\langle 0\right| + c_2^{z*}\cdot\left\langle 1\right|\right) Z \left(c_1^{z}\cdot\left|0\right\rangle + c_2^{z}\cdot\left|1\right\rangle\right) = \left|c_1^{z}\right|^2 - \left|c_2^{z}\right|^2,$$

$$\left\langle\psi\right| X \left|\psi\right\rangle = \left(c_1^{x*}\cdot\left\langle +\right| + c_2^{x*}\cdot\left\langle -\right|\right) X \left(c_1^{x}\cdot\left|+\right\rangle + c_2^{x}\cdot\left|-\right\rangle\right) = \left|c_1^{x}\right|^2 - \left|c_2^{x}\right|^2,$$

$$\left\langle\psi\right| Y \left|\psi\right\rangle = \left(c_1^{y*}\cdot\left\langle +i\right| + c_2^{y*}\cdot\left\langle -i\right|\right) Y \left(c_1^{y}\cdot\left|+i\right\rangle + c_2^{y}\cdot\left|-i\right\rangle\right) = \left|c_1^{y}\right|^2 - \left|\right.$$

Using the inner products

Here we have taken into account that the inner product of orthonormal vectors is 0 (e.g. $\langle 0|\,|1\rangle = 0$, $\langle +|\,|-\rangle = 0$, $\langle +i|\,|-i\rangle = 0$).
But what are these $|c|^2$s? The $|c_1^z|^2$ and $|c_2^z|^2$ are by definition the probabilities that after Z basis measurement (measuring is it $|0\rangle$ or is it $|1\rangle$) the quantum state $|\psi\rangle$ will become $|0\rangle$ or $|1\rangle$ respectively.

Rethinking the basis

In order to find that value, we should run our program with our trial $|\psi\rangle$ wavefunction and do Z measurement on the qubit N times (it is named *shots* in the code).
The probability of finding the qubit after measurment in $|0\rangle$ state will be equal to $|c_1^z|^2 = \frac{n_0}{N}$, where n_0 is the number of the $|0\rangle$ state measurments. Similarly, $|c_2^z|^2 = \frac{n_1}{N}$, where n_1 is the number of the $|1\rangle$ state measurments.
Thus, the final expectation value will be $\langle Z \rangle = \frac{n_0 - n_1}{N}$.

Measurements

For $\langle X \rangle = \frac{n_+ - n_-}{N}$ and $\langle Y \rangle = \frac{n_{+i} - n_{-i}}{N}$ the expectation value estimation procedure stays the same.
Here n_+ and n_- are numbers of measurements in X basis that corresponds to $|+\rangle$ or $|-\rangle$ outcomes respectively. And n_{+i} and n_{-i} are numbers of measurements in Y basis that corresponds to $|+i\rangle$ or $|-i\rangle$ outcomes respectively.

Computational basis

The difficulty comes from the fact that one may have the possibility to measure only in the Z basis. To solve this difficulty we still do a Z basis measurement, but, before that, we apply specific operators to the $|\psi\rangle$ state.
We try to apply such an operator that after measuring the probability of $|0\rangle$ outcome will be equal to the probability of $|+\rangle$ ($|+i\rangle$) outcome.
And the probability of $|1\rangle$ outcome will be equal to the probability of $|-\rangle$ ($|-i\rangle$) outcome.

Unitary transformation of $\boldsymbol{X}$

If we use the Hadamard gate

$$\boldsymbol{H} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

we can rewrite

$$\boldsymbol{X} = \boldsymbol{HZH}.$$

The Hadamard gate/matrix is a unitary matrix with the property that $\boldsymbol{H}^2 = \boldsymbol{I}$.

Generalizing

For the one-qubit Hamiltonian we have toyed with till now, we can thus rewrite in an easy way the Hamiltonian so that we can perform measurements using our favorite computational basis.
The transformation of the Pauli $\boldsymbol{X}$ matrix can be generalized, as we will see in more detail next week for the two-qubit Hamiltonian and the Lipkin model, to the following expression

$$\mathcal{P} = \boldsymbol{U}^{\dagger}\boldsymbol{M}\boldsymbol{U},$$

where $\mathcal{P}$ represents some combination of the Pauli matrices and the identity matrix, $\boldsymbol{U}$ is a unitary matrix and $\boldsymbol{M}$ represents the gate/matrix which performs the measurements, often represented by a Pauli $\boldsymbol{Z}$ gate/matrix.

Implementing the VQE

For a one-qubit system we can reach every point on the Bloch sphere (as discussed earlier) with a rotation about the x-axis and the y-axis.
We can express this mathematically through the following operations (see whiteboard for the drawing), giving us a new state $|\psi\rangle$

$$|\psi\rangle = R_y(\phi)R_x(\theta)|0\rangle.$$

Multiple ansatzes

We can produce multiple ansatzes for the new state in terms of the angles θ and ϕ. With these ansatzes we can in turn calculate the expectation value of the above Hamiltonian, now rewritten in terms of various Pauli matrices (and thereby gates), that is compute

$$\langle\psi|(c+\mathcal{E})\boldsymbol{I}+(\Omega+\omega_z)\boldsymbol{\sigma}_z+\omega_x\boldsymbol{\sigma}_x|\psi\rangle.$$

Rotations again

We can now set up a series of ansatzes for $|\psi\rangle$ as function of the angles θ and ϕ and find thereafter the variational minimum using for example a gradient descent method.
To do so, we need to remind ourselves about the mathematical expressions for the rotational matrices/operators.

$$R_x(\theta) = \cos\frac{\theta}{2}\boldsymbol{I} - \imath\sin\frac{\theta}{2}\boldsymbol{\sigma}_x,$$

and

$$R_y(\phi) = \cos\frac{\phi}{2}\boldsymbol{I} - \imath\sin\frac{\phi}{2}\boldsymbol{\sigma}_y.$$

Simple code

```
# define the rotation matrices
# Define angles theta and phi
theta = 0.5*np.pi; phi = 0.2*np.pi
Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
Ry = np.cos(phi*0.5)*I-1j*np.sin(phi*0.5)*Y
#define basis states
basis0 = np.array([1,0])
basis1 = np.array([0,1])

NewBasis = Ry @ Rx @ basis0
print(NewBasis)
# Compute the expectation value
#Note hermitian conjugation
Energy = NewBasis.conj().T @ Hamiltonian @ NewBasis
print(Energy)
```

Not an impressive results. We set up now a loop over many angles θ and ϕ and compute the energies

```
# define a number of angles
n = 20
angle = np.arange(0,180,10)
n = np.size(angle)
ExpectationValues = np.zeros((n,n))
for i in range (n):
    theta = np.pi*angle[i]/180.0
    Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
    for j in range (n):
```

Gradient descent and calculations of gradients

In order to optimize the VQE ansatz, we need to compute derivatives with respect to the variational parameters. Here we develop first a simpler approach tailored to the one-qubit case. For this particular case, we have defined an ansatz in terms of the Pauli rotation matrices.

Setting up gradients

These define an arbitrary one-qubit state on the Bloch sphere through the expression

$$|\psi\rangle = |\psi(\theta, \phi)\rangle = R_y(\phi)R_x(\theta)|0\rangle.$$

Each of these rotation matrices can be written in a more general form as

$$R_i(\gamma) = \exp -(\imath\frac{\gamma}{2}\sigma_i) = \cos(\frac{\gamma}{2})\boldsymbol{I} - \imath\sin(\frac{\gamma}{2})\boldsymbol{\sigma}_i,$$

where σ_i is one of the Pauli matrices $\sigma_{x,y,z}$.

Derivatives

It is easy to see that the derivative with respect to γ is

$$\frac{\partial R_i(\gamma)}{\partial \gamma} = -\frac{\gamma}{2}\boldsymbol{\sigma}_i R_i(\gamma).$$

Derivatives of the expectation value of the Hamiltonian

We can now calculate the derivative of the expectation value of the Hamiltonian in terms of the angles θ and ϕ. We have two derivatives

$$\frac{\partial}{\partial\theta}\left[\langle\psi(\theta,\phi)|\boldsymbol{H}|\psi(\theta,\phi)\rangle\right]=\frac{\partial}{\partial\theta}\left[\langle\boldsymbol{H}(\theta,\phi)\rangle\right]=\langle\psi(\theta,\phi)|\boldsymbol{H}(-\frac{i}{2}\boldsymbol{\sigma}_x|\psi(\theta,\phi)\rangle+$$

and

$$\frac{\partial}{\partial\phi}\left[\langle\psi(\theta,\phi)|\boldsymbol{H}|\psi(\theta,\phi)\rangle\right]=\frac{\partial}{\partial\phi}\left[\langle\boldsymbol{H}(\theta,\phi)\rangle\right]=\langle\psi(\theta,\phi)|\boldsymbol{H}(-\frac{i}{2}\boldsymbol{\sigma}_y|\psi(\theta,\phi)\rangle-$$

Two addtional expectation values

This means that we have to calculate two additional expectation values in addition to the expectation value of the Hamiltonian itself. If we stay with an ansatz for the single qubit states given by the above rotation operators, we can, following for example the article by Maria Schuld et al, show that the derivative of the expectation value of the Hamiltonian can be written as (we focus only on a given angle ϕ)

$$\frac{\partial}{\partial\phi}\left[\langle \boldsymbol{H}(\phi)\rangle\right] = \frac{1}{2}\left[\langle \boldsymbol{H}(\phi+\frac{\pi}{2})\rangle - \langle \boldsymbol{H}(\phi-\frac{\pi}{2})\rangle\right].$$

Rotations again and again

To see this, consider again the definition of the rotation operators. We can write these operators as

$$R_i(\phi) = \exp -\imath(\phi \boldsymbol{\sigma}_i),$$

with ***sigma***$_i$, with $\boldsymbol{\sigma}_i$ being any of the Pauli matrices X, Y and Z. The latter can be generalized to other unitary matrices as well. The derivative with respect to ϕ gives

$$\frac{\partial R_i(\phi)}{\partial \phi} = -\frac{\imath}{2}\boldsymbol{\sigma}_i \exp -\imath(\phi \boldsymbol{\sigma}_i) = -\frac{\imath}{2}\boldsymbol{\sigma} R_i(\phi).$$

Bloch sphere math

Our ansatz for a general one-qubit state on the Bloch sphere contains the product of a rotation around the x-axis and the y-axis. In the derivation here we focus only on one angle however. Our ansatz is then given by

$$|\psi\rangle = R_i(\phi)|0\rangle,$$

and the expectation value of our Hamiltonian is

$$\langle\psi|\hat{H}|\psi\rangle = \langle 0|R_i(\phi)^\dagger \hat{H} R_i(\phi)|0\rangle.$$

Derivatives

Our derivative with respect to the angle ϕ has a similar structure, that is

$$\frac{\partial}{\partial \phi}\left[\langle \psi(\theta,\phi)|\boldsymbol{H}|\psi(\theta,\phi)\rangle\right] = \langle \psi(\theta,\phi)|\boldsymbol{H}(-\frac{\imath}{2}\boldsymbol{\sigma}_y|\psi(\theta,\phi)\rangle + \text{ h.c.}$$

Rewriting

In order to rewrite the equation of the derivative, the following relation is useful

$$\langle\psi|\hat{A}^\dagger\hat{B}\hat{C}|\psi\rangle = \frac{1}{2}\Big[\langle\psi|(\hat{A}+\hat{C})^\dagger\hat{B}(\boldsymbol{A}+\hat{C})|\psi\rangle - \langle\psi|(\hat{A}-\hat{C})^\dagger\hat{B}(\boldsymbol{A}-\hat{C})|\psi\rangle$$

where $\hat{A}$, $\hat{B}$ and $\hat{C}$ are arbitrary hermitian operators.

Final manipulations

If we identify these operators as $\hat{A} = \boldsymbol{I}$, with $\boldsymbol{I}$ being the unit operator, $\hat{B} = \hat{H}$ our Hamiltonian, and $\hat{C} = -\imath\boldsymbol{\sigma}_i/2$, we obtain the following expression for the expectation value of the derivative (excluding the hermitian conjugate)

$$\langle\psi|\boldsymbol{I}^\dagger\hat{H}(-\frac{\imath}{2}\boldsymbol{\sigma}_i|\psi\rangle = \frac{1}{2}\Big[\langle\psi|(\boldsymbol{I} - \frac{\imath}{2}\boldsymbol{\sigma}_i)^\dagger\hat{H}(\boldsymbol{I} - \frac{\imath}{2}\boldsymbol{\sigma}_i)|\psi\rangle - \langle\psi|(\boldsymbol{I} + \frac{\imath}{2}\boldsymbol{\sigma}_i)^\dagger\hat{H}(\boldsymbol{I}$$

The expressions to implement

If we then use that the rotation matrices can be rewritten as

$$R_i(\phi) = \exp -(\imath\frac{\phi}{2}\sigma_i) = \cos(\frac{\phi}{2})\boldsymbol{I} - \imath\sin(\frac{\phi}{2})\boldsymbol{\sigma}_i,$$

we see that if we set the angle to $\phi = \pi/2$, we have

$$R_i(\frac{\pi}{2}) = \cos(\frac{\pi}{4})\boldsymbol{I} - \imath\sin(\frac{\pi}{4})\boldsymbol{\sigma}_i = \frac{1}{\sqrt{2}}\left(\boldsymbol{I} - \frac{\imath}{2}\boldsymbol{\sigma}_i\right).$$

Final expression

This means that we can write

$$\langle\psi|\boldsymbol{I}^{\dagger}\hat{H}(-\frac{\imath}{2}\boldsymbol{\sigma}_i|\psi\rangle = \frac{1}{2}\Big[\langle\psi|R_i(\frac{\pi}{2})^{\dagger}\hat{H}R_i(\frac{\pi}{2})|\psi\rangle - \langle\psi|R_i(-\frac{\pi}{2})^{\dagger}\hat{H}R_i(-\frac{\pi}{2})^{\dagger}|\psi$$

Basics of gradient descent and stochastic gradient descent

In order to implement the above equations, we need to remind the reader about basic elements of various optimization approaches. Our main focus here will be various gradient descent approaches and quasi-Newton methods like Broyden's algorithm and variations thereof.
This material is covered by the lectures from FYS4411 on gradient optimization

Computing quantum gradients

Let us implement efficient implementations of gradient methods to the derivatives of the Hamiltonian expectation values.

```
from  matplotlib import pyplot as plt
import numpy as np
from scipy.optimize import minimize
dim = 2
Hamiltonian = np.zeros((dim,dim))
e0 = 0.0
e1 = 4.0
Xnondiag = 0.20
Xdiag = 3.0
Eigenvalue = np.zeros(dim)
# setting up the Hamiltonian
Hamiltonian[0,0] = Xdiag+e0
Hamiltonian[0,1] = Xnondiag
Hamiltonian[1,0] = Hamiltonian[0,1]
Hamiltonian[1,1] = e1-Xdiag
# diagonalize and obtain eigenvalues, not necessarily sorted
EigValues, EigVectors = np.linalg.eig(Hamiltonian)
permute = EigValues.argsort()
EigValues = EigValues[permute]
# print only the lowest eigenvalue
print(EigValues[0])

# Now rewrite it in terms of the identity matrix and the Pauli matrix
X = np.array([[0,1],[1,0]])
Y = np.array([[0,-1j],[1j,0]])
```

A smarter way of doing this

The above approach means that we are setting up several matrix-matrix and matrix-vector multiplications. Although straight forward it is not the most efficient way of doing this, in particular in case the matrices become large (and sparse). But there are some more important issues.
In a physical realization of these systems we cannot just multiply the state with the Hamiltonian. When performing a measurement we can only measure in one particular direction. For the computational basis states which we have, $|0\rangle$ and $|1\rangle$, we have to measure along the bases of the Pauli matrices and reconstruct the eigenvalues from these measurements.

The code for the one qubit case (code developed by August Gude, 2023)

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set_theme(font_scale=1.5)
from tqdm import tqdm

sigma_x = np.array([[0, 1], [1, 0]])
sigma_y = np.array([[0, -1j], [1j, 0]])
sigma_z = np.array([[1, 0], [0, -1]])
I = np.eye(2)

def Hamiltonian(lmb):
    E1 = 0
    E2 = 4
    V11 = 3
    V22 = -3
    V12 = 0.2
    V21 = 0.2

    eps = (E1 + E2) / 2
    omega = (E1 - E2) / 2
    c = (V11 + V22) / 2
    omega_z = (V11 - V22) / 2
    omega_x = V12

    H0 = eps * I + omega * sigma_z
    H1 = c * I + omega_z * sigma_z + omega_x * sigma_x
```

Plans for next week

1. Introducing the Lipkin model
2. Solving problems with more than one qubit and discussion of project 1